guard-work-track-binding.sh

Revision: 2 **Last modified:** 2026-07-10T21:50:00Z **Authority:** constitution §11.4.191 (work-to-track/branch binding enforcement) **Maintainer:** constitution submodule (inherited by reference per §11.4.177 / §11.4.28(B)) **Scope:** §11.4.18 companion doc for `constitution/scripts/hooks/guard-work-track-binding.sh`

Overview

`guard-work-track-binding.sh` is a Claude Code **PreToolUse guard hook** — the PREVENTIVE layer of §11.4.191. It makes a **mis-track commit** or a **mis-track dispatch** impossible to *issue* at the tool-call boundary, independent of any agent's recall (§11.4.109 anti-forgetting).

§11.4.191 is the file-**placement** generalisation of §11.4.181 (branch-**name** consistency). The sibling `guard-branch-consistency.sh` fires only on a feature-branch *create*, so committing a logic-group's files onto an already-existing branch (the forensic scrcpy → `main` mis-landing, 2026-07-04) slipped past it. This hook closes exactly that gap.

The hook is thin: it detects the tool class, resolves the current/declared `(branch, track)`, computes the impacted work (changed files or a ticket) and delegates the actual binding decision to the shared resolver `multitrack_work_binding.sh` (single code path, two entry points — this hook + the commit wrapper / pre-build gate).

Prerequisites

- `sqlite3` on `PATH` (via the resolver; absent ⇒ fail-closed BLOCK, §11.4.6).
- `git` (only for the commit-path branch/file-set derivation).

- The §11.4.93/§11.4.95 workable-items DB (schema **v4**: `logic_groups.canonical_track`
  - the `group_paths` file-scope manifest). Resolved by `resolve_workable_items_db` (see "Submodule-context DB resolution" below) — `$WI_DB` wins outright; otherwise the guard walks UP from `git rev-parse --show-toplevel` through every nested submodule → superproject link (`git rev-parse --show-superproject-working-tree`) so a commit issued from INSIDE a submodule (per §11.4.26) resolves against the PARENT superproject's registry, never a submodule-relative path where the DB does not live.
- Wired as a `PreToolUse` hook under BOTH the `Bash` and `Agent|Task|TaskCreate` matchers in `.claude/settings.json` (a later, consumer-side task — this hook ships the mechanism, not the wiring).

Usage

It is not run by hand — Claude Code invokes it, passing the tool invocation as JSON on stdin. Exit `0` = allow, exit `2` = BLOCK (stderr text is fed back to Claude as the refusal reason).

Manual smoke:

```
echo '{"tool_name":"Agent","tool_input":{"description":"(T1/main - claude4) ATM-900 scrcpy"}}' \
  | WI_DB=docs/workable_items.db bash constitution/scripts/hooks/guard-work-track-binding.sh ;
            echo $?
                                0
```

Scope (two handled classes; every other tool ⇒ exit)

- **`Bash`** — a git-commit-class command (`git commit` , `scripts/commit_all.sh` , `commit_docs.sh`). The to-be-committed file set is read from git **plumbing** (`git diff --cached --name-only` ; for a `commit_all` -class *broad stage* the whole dirty worktree via `git status --porcelain`), then checked against the current branch (`git rev-parse --abbrev-ref HEAD`) + track (cwd `/mnt/track<N>`).

- **`Agent | Task | TaskCreate`** — an agent dispatch. The §11.4.182 `(T<N>/<branch>[ - <alias>])` label + the `\b(ATM|SPK)-[0-9]+\b` ticket(s) in the `description` (fallback `subagent`) are checked against the ticket's group's canonical `(branch, track)`.

Behaviour matrix

Situation	Result
A `group_paths`-owned file staged on a branch ≠ its group's `destination`	**BLOCK (exit 2)**
A ticket dispatched onto a label `(branch, track)` ≠ its group's canonical pair	**BLOCK (exit 2)**
Correctly-placed commit / dispatch	allow (exit 0)
Unclassified file (no glob owner) / unticketed dispatch	allow (exit 0) — main-eligible, honest partial coverage (§11.4.3)
Registry unreadable (no `sqlite3` / no DB)	**fail-closed BLOCK (exit 2)** (§11.4.6)
Ambiguous file scope (two groups claim one path at equal glob length)	**fail-closed BLOCK (exit 2)**
Dynamic `$(…)`/backtick commit path-spec	**fail-closed BLOCK (exit 2)**
`# guardrails:allow <reason>` on a commit command	audited WARN → allow (exit 0)
Dispatch with an absent/malformed label	allow (exit 0) — the sibling `guard-track-branch-label.sh` BLOCKs that (single responsibility)
Any non-Bash / non-Agent tool	allow (exit 0)

Edge cases

- **Unborn branch:** on a repo with no commits `git rev-parse --abbrev-ref HEAD` returns `HEAD`; the hook then compares against `HEAD` (a real repo always has commits, so this is a fixture-only concern).
- **Alias suffix:** a label field `feature/mistiq-vader - deepseek` strips to branch `feature/mistiq-vader` (a git ref never contains a space, so any `-` is the §11.4.182 alias separator).
- **Track unknown (`?` / off- `/mnt/trackN`):** the track assertion is SKIPPED (honest — an unknown track is not a proven mismatch, §11.4.6); the branch assertion still runs.
- **Pre-v4 DB (no `group_paths`):** the file-path enforcement is inert (no glob owners) — branch-enforced-only until the manifest lands; ticket resolution still works.

Submodule-context DB resolution (§ . . / § . .)

When this guard's own cwd is INSIDE a submodule (e.g. this constitution submodule, committed per §11.4.26), the shared resolver's own auto-discovery binds `git rev-parse --show-toplevel` to the SUBMODULE's innermost root — where `docs/workable_items.db` does not exist — so it fail-closed BLOCKed every legitimate submodule commit unconditionally, regardless of file-scope classification. `resolve_workable_items_db` closes this: `$WI_DB` (if set) wins verbatim (unchanged precedence + unchanged fail-closed-on-absent behaviour); otherwise it walks UP from `git rev-parse --show-toplevel` through every `git rev-parse --show-superproject-working-tree` hop (depth-bounded to 10), probing each root OUTERMOST-first for `docs/workable_items.db` / `docs/.workable_items.db`. The guard passes the found path as an explicit `--db <path>` to the resolver at BOTH call sites; when NOTHING is found anywhere, `--db` is omitted and the resolver's own unchanged auto-discovery + fail-closed BLOCK applies exactly as before (§11.4.6 — no weakening of the fail-closed guarantee for the genuinely-no-DB case). This function itself NEVER fails or blocks — it only WIDENS where a real registry is discovered; the resolver alone owns the fail-closed BLOCK decision.

Honest boundary (§ . .)

Best-effort prevention, NOT a security boundary. The file set is derived from git **plumbing**, never argv-parsed, so it is never *under*-blocking (a pathspec that commits a subset can only make the hook *over*-strict, which the escape hatch clears). A raw `git` invoked from inside a script the hook never parses is caught by the DETECTIVE pre-build gate `CM-WORK-TRACK-BINDING-ENFORCED`, which inspects the real committed diff regardless of HOW the commit happened — that gate is the enforcement boundary; this hook is defense-in-depth (same split as `guard-branch-consistency.sh`).

Internal behaviour

1. Parse `tool_name` (jq-preferred, awk fallback — no jq dependency).
2. Dispatch path → parse the §11.4.182 label + tickets → resolver `check --branch <B_lbl> --track <T_lbl> --ticket …`.
3. Commit path → detect commit intent (argv-segment-aware) + broad-stage + dynamic-pathspec → resolve the plumbing file set → resolver `check --branch <cur> --track <cur> --files-from -`.
4. Resolver non-zero ⇒ BLOCK (or WARN under `# guardrails:allow`).

Related scripts

- `multitrack_work_binding.sh` — the shared resolver this hook calls.
- `guard-branch-consistency.sh` — the §11.4.181 create-time sibling (branch NAME).
- `guard-track-branch-label.sh` — the §11.4.182 label-FORMAT sibling.
- `test_guard_work_track_binding.sh` — the hermetic test suite (34 cases, both paths; section D covers submodule-context DB resolution).

Testing

```
bash constitution/scripts/hooks/test_guard_work_track_binding.sh
```

Hermetic: builds a throwaway schema-v4 registry + a throwaway git repo (with a real seed commit); exercises BLOCK + ALLOW + fail-closed on both paths; exits 1 if any case fails. The paired §1.1 mutation that strips the resolver's load-bearing branch comparison lands in the consumer meta-test (a separate work item per §11.4.191(§6)).

Last verified: 2026-07-10 — `bash -n` + `sh -n` clean (§11.4.67); hermetic suite 34/34 PASS (incl. section D: submodule-context DB resolution — parent DB enforced, fail-closed preserved); the resolver branch-comparison mutation flips a planted mis-placement from BLOCK to ALLOW (clause proven load-bearing).